

Both the coefficients in one query: A combined select query for the slope m and the y-intercept c can be written as follows:

```
SELECT
    ROUND(    #m
        SUM(
            (Age-(SELECT SUM(Age)/ COUNT(Age) AS avgAge FROM
carprice))*
            (Price-(SELECT SUM(Price)/ COUNT(Price) AS avgPrice
FROM carprice))
        )
    / SUM(
        (Age-( SELECT SUM(Age)/ COUNT(Age) AS avgAge FROM
carprice)) *
        (Age-( SELECT SUM(Age)/ COUNT(Age) AS avgAge FROM
carprice))
    ) ,2)
    AS m,

    ROUND((SUM(Price)/ COUNT(Price)) -    #c
        (SUM(
            (Age-(SELECT SUM(Age)/ COUNT(Age) AS avgAge FROM
carprice))*
            (Price-(SELECT SUM(Price)/ COUNT(Price) AS
avgPrice FROM carprice))
        )
    / SUM(
        (Age-(SELECT SUM(Age)/ COUNT(Age) AS avgAge FROM
carprice)) *
        (Age-(SELECT SUM(Age)/ COUNT(Age) AS avgAge FROM
carprice))
    ) ) *
        (SUM(Age)/ COUNT(Age)),2)    AS c
FROM carprice
```

Like the earlier one, this query will also return -.50 and 7.84 for m and c, respectively.

The output is:

m	c
-0.50	7.84

Estimate an unknown value

From the above three queries, we have seen how this complex query can be implemented in a step by step way. If the task is to implement Equation 1 directly in one go, then the equation can be written in SQL as follows:

```
set @testage=3
SELECT @testage*ROUND(    #xm
```

```
    SUM(
        (Age-(SELECT SUM(Age)/ COUNT(Age) AS avgAge FROM
carprice))*

        (Price-(SELECT SUM(Price)/ COUNT(Price) AS avgPrice
FROM carprice))
    )

    / SUM(
        (Age-( SELECT SUM(Age)/ COUNT(Age) AS avgAge FROM
carprice)) *
        (Age-( SELECT SUM(Age)/ COUNT(Age) AS avgAge FROM
carprice))
    ) ,2) +
    ROUND(    #3
        (SUM(Price)/ COUNT(Price)) -
        (SUM(
            (Age-(SELECT SUM(Age)/ COUNT(Age) AS avgAge
FROM carprice))*
            (Price-(SELECT SUM(Price)/ COUNT(Price) AS
avgPrice FROM carprice))
        ) /
        SUM(
            (Age-(SELECT SUM(Age)/ COUNT(Age) AS
avgAge FROM carprice)) *
            (Age-(SELECT SUM(Age)/ COUNT(Age) AS avgAge
FROM carprice))
        ) ) *
        (SUM(Age)/ COUNT(Age)),2) as y
FROM carprice
```

The age of a given car can be provided by defining an SQL variable @testage at the outset of the query as shown in the above example. After execution of this query, you will get 6.84 as the result of this query.

The output is:

y
6.84

For beginners, this query may be a bit cumbersome but a little care while writing such complex queries will give interesting results, and a sense of the power of SQL queries. This power of an elegant query can be further extended with R-SQL or Python-SQL.